digithings · edition 2026 · version 1.0

DIGITHINGS

the digithings book

CONTENTS

01 Why this book exists 3

02 Software that lives anywhere 4

03 Open source, open core 5

04 How it scales 6

05 One environment for an AI application 7

06 What you can build 8

07 digigraph, the orchestration hub 10

08 digillm, one model layer 11

09 MCP, every capability as a tool 12

10 digisearch, finding what matters 13

11 digiquant, the research vertical 14

12 digivoice, the voice loop on your machine 15

13 digichat, the interface people touch 16

14 digikey, identity without trust 17

15 digitrace, seeing what the system did 18

16 The local stack, one machine 19

17 Start on your machine 20

01

Why this book exists

Most of what gets written about AI infrastructure is written by whoever rents it to you. This comes from the other side of that arrangement.

What follows is what digithings believes about building AI software, the principles the codebase follows, and what that makes possible for the people who use it. Setup and every endpoint live in the documentation; the reasoning behind those decisions lives here.

What we hold to

Own the stack. The software that runs your work should be software you can read, run and change. Renting is a choice you should be able to make later, not a condition of starting.

Location is a deployment decision. The same application runs on a laptop, on a server you own and in a cloud account. Moving it changes configuration, not code.

Open by default. The components are open source. The boundaries between them are open protocols, so no part of the stack can hold the rest hostage.

Small parts, one job each. Model access, orchestration, retrieval, identity, tracing and the interface are separate modules. You run the ones the work needs and leave the rest switched off.

Grow in place. A solo builder and an organisation use the same contracts. Capability and cost are dials you turn up as the work proves itself.

Show the working. Answers cite their sources, research results keep their provenance, and every request can leave a trail. Trust comes from what the system records, not from what it claims.

How to read it

The case comes first: software that can live anywhere, an open core, a stack that scales without a rewrite. Then the modules themselves, in the order the work arrives. It ends where every project should start, on your own machine, with the first thing to build.

It is written for whoever has to decide where their AI software runs, and for whoever maintains it afterwards. The argument holds the same for one machine or a thousand.

Read it through in one sitting, or open the chapter your question lives in.

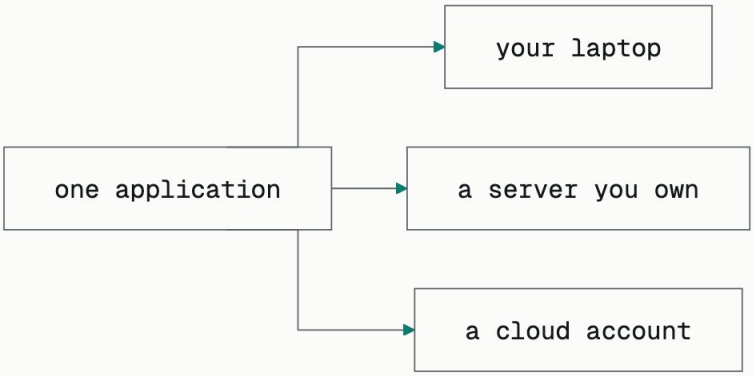
02

Software that lives anywhere

An AI application should not be tied to the machine it first ran on. Over its life the model endpoint changes, the data grows and the number of people using it goes from one to many. The application should not have to change with them.

Most stacks decide where your software lives on day one. A hosted platform needs an account before the first request. A framework built around one vendor's runtime only runs there. By the time you know what the application needs, the choice has been made for you, and undoing it is a migration project.

digithings takes the opposite position. The same code runs on a laptop, on a server you own and in a cloud account, with the same service contracts, the same setting names and the same identity model in each. You build locally, prove the application end to end on real data, and move it when the work needs more capacity. The move is a deployment choice, not a rewrite.



one application, three homes: the contracts between modules are the same in each

This is not an argument against the cloud. A cloud account is often the right home for a production service, and digithings runs there well. The argument is against choosing infrastructure before you know what the application needs, and against infrastructure that makes the choice hard to revisit.

Three things keep the choice open. The code is open source, so nothing is hidden behind an account. The seams between modules are standard protocols, so any module can run on a different machine from its neighbours. And no module assumes where the others live: each finds its neighbours through configuration, never through a hard-coded address.

The result is that location stops being an architectural commitment. Stay on one machine while the work is small. Move to a cloud account when it needs more capacity. Split one module onto its own hardware when it grows faster than the rest. At every step it is the code you started with.

03

Open source, open core

digithings is open core. The components you would build an AI application from are published under the MIT licence: the orchestration hub, the model layer, retrieval, backtesting and optimisation, the chat interface, identity, tracing and the shared libraries. You can read them, run them, change them and ship them inside your own product.

Why open

Open source is usually argued on price. The stronger reasons are practical.

You can inspect it. When an answer looks wrong, you can follow the request through code you can read, down to the line that made the decision. A closed component turns that question into a support ticket.

You can move it. Portability depends on knowing every assumption a component makes about where it runs. In a closed component, those assumptions are exactly what you cannot see.

It outlives the vendor. Software you build on keeps working whatever happens to the company that wrote it. The code is in your hands, under a licence that does not expire.

Open at the seams

Open code is not enough if the boundaries are proprietary. A stack can publish every module and still lock you in through the way they talk to each other.

digithings keeps its seams on standards that existed before it and will exist after it. Models are reached through the chat format most providers and local runners already speak. Tools are described with the Model Context Protocol, so any client that speaks it can use them. Identity is carried in standard signed credentials that any service can verify. Replace one module with your own and the others do not notice.

What stays closed

The closed part is narrow and has a rule. Anything a person would reasonably want to self-host for their own work stays open. What is closed is the commercial layer built on top: the finance workflows that run research cycles and portfolio decisions, the layer that turns research into live orders, the machinery for running many tenants on one deployment, and managed hosting.

That line can move as the project grows. The rule behind it does not.

04

How it scales

"Does it scale?" means one thing for a rented service and another for a stack you run. A rented service scales when someone raises a limit on a dashboard, and the bill follows. A stack you run scales when you decide what to add and where. That decision is yours, and so is the work of running what you add. The stack is built to keep that work small enough for one person to carry.

Two kinds of scale

The first kind is load: more people, more requests, more documents. That is solved by running more copies of a module, or by moving it to bigger hardware. Modules keep their state in stores built for it, so adding capacity does not change how they work.

The second kind is capability: the application learns to do more. A chat assistant gains retrieval, then a research pipeline, then tools that act on other systems. Each step is one more module switched on.

Three dials

Which modules run. Start with the hub, the model layer and the chat interface. Add retrieval, research, voice, identity or tracing when the work calls for them. A module you do not run costs nothing.

Which models answer. A small local model is free per call and private. A large hosted model reasons better and costs per token. The model layer makes this a routing setting, so different tasks can use different models at once.

Where it runs. One machine, a server you own, a cloud account, or a mix. Each module can sit wherever its workload is cheapest.

Who	What they typically run	Where the cost goes
A solo builder or researcher	Hub, model layer, chat and one vertical, on one machine with local models	The machine they own; hosted models only by choice
A team	Shared retrieval, a research pipeline and identity, on one server or a small cloud account	One server, plus models for the work that needs them
An organisation	Every module, tracing on all requests, several machines or a cloud account	Capacity sized per module, models routed per task

The rows describe one system at three sizes. The code, the contracts and the setting names are the same in each, and the application built in the first row is the one running in the last. The ceiling is set by hardware and judgement.

05

One environment for an AI application

An AI application is more than a model call. It needs a way to reach models, route work between them, find the right context, call tools, identify who is asking, show an interface and explain afterwards what happened. Most projects spend their first months rebuilding that scaffolding before they get to the part that is actually theirs.

digithings is that scaffolding, built once. Each module has one job and a published contract, so you can run one of them, a few, or all of them.

Module	Its one job
digigraph	The hub. Routes each request to the right tools and streams the answer.
digillm	The model layer. One way for every service to call any model.
MCP	The tool protocol. Every capability is a tool any client can discover.
digisearch	Retrieval. Indexes your documents and finds the passages that matter.
digivault	Documents. A linked markdown vault, readable by people and tools.
digiquant	Research. Data, strategies, backtests and optimisation, in order.
digivoice	Voice. Dictation, rewriting and speech, all on your machine.
digichat	The interface. Where people ask, read answers and see artifacts.
digikey	Identity. Short-lived credentials every service checks itself.
digitrace	Tracing. A record of what every request did.
digibase	The shared base. Error shapes, request ids, service sign-in.

How the parts fit

Requests come in through the interface or an outside caller and go to the hub. The hub asks the verticals what tools they offer, runs the ones the request needs and reaches models through the model layer. Identity can be checked at every boundary and every step traced. None of it is fixed: any part can be replaced by one of your own that keeps the same contract. After the use cases, the chapters take the main parts one at a time.

06

What you can build

The same modules make different applications. A use case is a choice of modules, not a separate product, and each of them starts on a single machine.

Financial research

A research desk needs market data, a way to test ideas against history, and results it can defend. Most tooling for this is either a spreadsheet that cannot be audited or a closed platform whose methodology you take on trust.

digiquant gives the work an ordered pipeline: data, strategy validation, backtest, optimisation and export, each stage reading only what the stage before it wrote. digigraph routes questions to it, digisearch brings in the filings and notes around a position, and digichat is where the analyst asks and reads. Every result carries its provenance, so an analyst can defend a number to someone who doubts it.

On one machine this is a private research environment for a single analyst. For a team, the pipeline moves to a shared server, backtests run on a timetable, and identity decides who can see which research.

Voice

Speech is the input people are least willing to share, and the usual way to process it is to stream it to someone else's servers. digivoice keeps the whole loop local: capture, transcription, an optional rewrite for where the text is going, and speech back. Nothing leaves the machine.

Because the output is ordinary text, voice becomes a way into the rest of the stack. A finished transcript pastes into whatever is in focus: a question in the chat, a research brief, made out loud while the hands are busy. Voice grows differently from the other three: each person runs a loop on their own machine, and there is nothing central to size.

Retrieval over your own documents

Most of what a team needs answered sits in its own files: contracts, notes, past research. Retrieval brings it to the model: your documents are split into passages, indexed by meaning, and the passages that matter are handed to the model with the question. The answer cites what it stands on, so a reader can check it.

digisearch owns the two halves, ingestion and answering, and keeps them apart. digivault holds documents as linked markdown, and digillm connects whichever model you choose. A workspace filter can travel with each query, so one index can hold several teams' documents. The same pipeline runs against a folder on a laptop and against a department's archive on managed storage.

Agents and exploration

An agent needs tools, a loop that decides what to do next, a way to ask a person before doing anything risky, and a record of what it did. digigraph runs the loop, MCP describes the tools, a per-server gate decides what may run unattended, and digitrace keeps the trail.

Agents gain most from sharing one environment. The first agent is usually a handful of scripts a team already has, each given a name, an input schema and a description. Outside agents and editors connect to the same tools through the same protocol, with the same permissions. Exploring a new idea means adding a tool, not building a new agent framework around it.

At a glance

Use case	Modules it runs	Starts on	Grows into
Financial research	digiquant, digigraph, digisearch, digichat	One analyst's machine	A shared research server with scheduled runs and access by role
Voice	digivoice, digichat	The machine with the microphone	Every machine where someone wants it, with no service to run
Retrieval	digisearch, digivault, digillm, digichat	A folder on a laptop	A department's archive on managed storage, with access rules per query
Agents	digigraph, MCP tools, digitrace	Your own tools, described over MCP	Many tool servers under one approval gate, shared with outside agents

One question, four modules

The use cases are not separate lanes. An analyst dictates a question about a holding, and digivoice turns it into text without the audio leaving the machine. In the chat, the hub asks the verticals what they offer: digisearch returns the filing passages behind the position, and digiquant reruns the backtest under the stated parameters. The answer cites both, and with tracing on, the trail shows every step that produced it.

Nothing in that chain was written for it. Each module did its one job, and the application is the set of parts that were switched on. The same chain runs unchanged when the research moves to a shared server, and that is how an idea proved on one machine grows without starting again.

07

digigraph, the orchestration hub

When a request arrives, something has to decide what happens next. A chat message, a research job and a dashboard widget all need the same loop: understand the request, pick the tools, run them in order and return the result. Without a hub, every caller writes that loop again and the copies drift apart. digigraph owns it once, so every interface shares one routing behaviour, one tool contract and one streaming format.

The hub is kept deliberately small. It routes, dispatches and streams, and leaves the real work to the verticals that own it.

How it works

digigraph runs each request through a state machine with stages for research, validation, backtesting and optional optimisation. Conditional edges decide which stages a given request needs. Around the machine sits a registry of tools. A few live inside the hub; the rest belong to the verticals, which publish descriptions of what they offer. The hub fetches those descriptions at runtime and never carries a hard-coded list.



the hub asks each vertical what it offers instead of assuming it

Because the hub asks, nothing has to be registered in advance. A vertical can ship a new tool without a new release of the hub, and a tool server the operator adds works as soon as it describes itself. A question about a company's filings, for example, reaches retrieval because retrieval said it can search documents, not because the hub was told where documents live.

Where it fits

The interfaces send the hub chat-shaped requests, and it answers in the streaming format ordinary chat clients already understand. Downstream it calls the verticals for retrieval and research, and models through the shared model layer. Identity arrives as a short-lived signed credential that the hub verifies itself rather than trusting the caller. With tracing on, every step is recorded, so a routing choice can be questioned after the fact.

The hub holds no list of where its neighbours live, so moving a vertical to another machine changes the hub's configuration and nothing else.

08

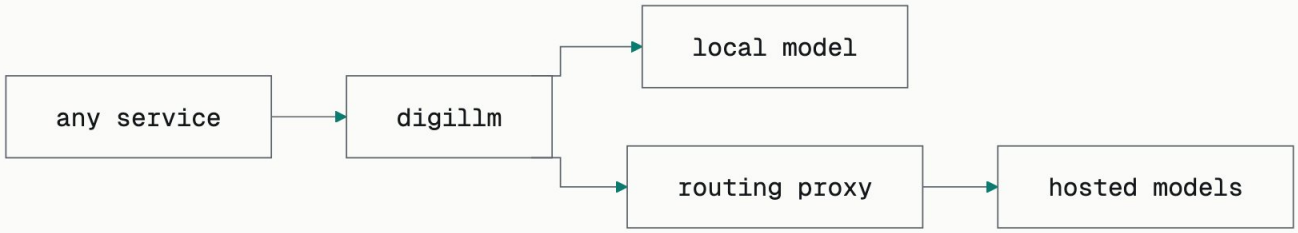
digillm, one model layer

Left alone, every service grows its own model client. Each picks a favourite vendor, retries in its own way and fails in its own way. A year later, switching models means touching every service, and nobody can say what a model call costs. digillm exists so that never happens: one library, used by every service, with no preference for any vendor.

With a shared layer, changing models is a configuration choice. digillm speaks the chat format that most providers, local runners and routing proxies already accept, so they all look the same to the calling code. Trying a new model changes where the traffic goes, not how the code asks for it.

How it works

digillm is a library, not a service. It ships inside the services that use it and defines the shapes every model call shares: the request, the tool call, the tool definition and structured answers checked against a schema. On top of those shapes it adds what every caller needs and nobody wants to write twice: retries, timeouts, usage accounting and optional tracing.



any service reaches any model through one layer; the provider is a setting

The library keeps almost no dependencies of its own and never imports a web framework. So the same code can live inside a chat interface, an orchestration hub and a research pipeline without dragging their worlds along.

Where it fits

The hub uses it for its own model calls, retrieval for web answers and checking passages, and research for usage records. Because the shapes are shared, a structured answer produced in one service can be read in another without translation. When tracing is configured, usage flows into the trail; when it is not, the hooks cost nothing.

A library has no home of its own. On a laptop it points at a local runner, in the cloud at whatever inference the deployment chose. Where models run stays an operator decision, never a code change.

09

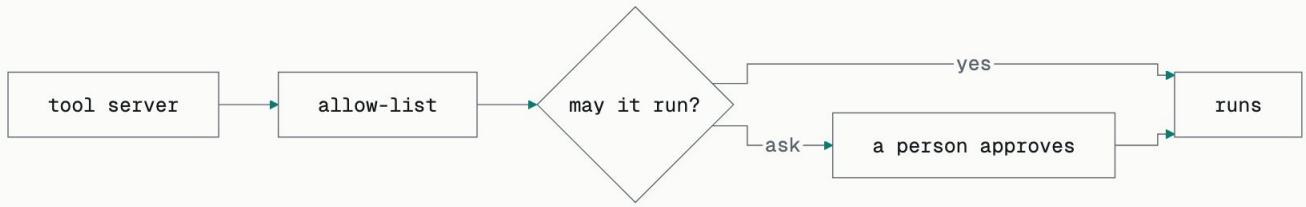
MCP, every capability as a tool

A system with dozens of capabilities needs one way to say what each of them does, readable by people and by models alike. The Model Context Protocol is that way: each tool has a name, a schema for its inputs and a description, offered over a standard transport. digithings uses it everywhere. Every capability the system offers is a described tool, and every tool a model can call passes the same allow-list.

The payoff is how new things plug in. A new search source, a new analysis or an integration with a team's tracker arrives as a tool description, and every part of the system already knows how to read one. Nobody patches the hub, the chat or the dashboard to adopt it. Once the description exists, the tool is integrated.

How it works

Tools live behind servers that advertise them. The hub lists what each server offers, filters the list through the operator's allow-list, and shows the model only what survives. A tool call carries the caller's identity, so it is authorised with the same scopes as any other request. A gate per server decides which tools may run unattended and which need a person to say yes first.



offered and executed are checked separately: a model only sees tools it may call

The gate is checked where the tool executes, not only where it is offered. A model is never shown a tool it could not call, and a call never relies on the offer alone. So a prompt that talks a model into trying a forbidden tool still fails at the gate.

Where it fits

The protocol is how tools reach the hub from outside it. Operator tool servers publish through it, and outside agents and code editors connect to the system's own tool servers through it. The verticals describe their tools to the hub in the same shape. Because the protocol does not care about transport, the contract is the same on one machine and across a network; only the list of connected servers changes.

10

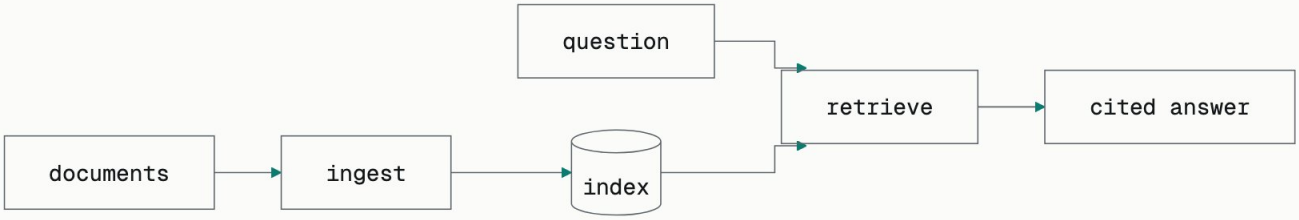
digisearch, finding what matters

Language models know a great deal and remember nothing of yours. Everything an organisation knows, its documents, its history, its earlier research, sits outside the model. digisearch brings it in. It takes in your documents, indexes them by meaning and finds the right passages when a question needs them.

Without retrieval a model answers from training memory: fluent, confident and sometimes invented. With it, the answer arrives with the passages it stands on, and a sceptical reader can check them. digisearch makes that the default shape of every answer that touches private knowledge.

How it works

Two paths stay strictly apart. Ingestion splits documents into passages and stores them in an index built for meaning, so a question about staff turnover finds a passage that only says attrition. Answering takes a question, retrieves the passages most likely to matter and hands them to the model with the question, so the model reasons over supplied evidence instead of recalling.



two paths share an index but never logic: indexing cannot change how a question is answered

The paths share storage and nothing else. A document being indexed cannot change how a question is answered, and answering never writes to the index. A workspace filter can ride along with every query, so one index can hold documents for several workspaces.

Where it fits

digisearch sits beside digiquant as one of the hub's research arms: quant work answers from market data, search answers from documents. The chat leans on it for grounded answers and the research pipeline calls it for background. Ingestion runs on demand, and either way it lands in the index the answering path reads.

The index backend is the only thing that changes between homes: an embedded index on a laptop, managed vector storage in the cloud, whatever fits the corpus on your own hardware. In every home, ingestion is an explicit act, so no document crosses a boundary you did not approve.

11

digiquant, the research vertical

Quantitative research has a failure mode most software does not: results that look rigorous and are not. A backtest that peeks at the future, steps run in a convenient order, a number nobody can reproduce. Each one manufactures confidence. digiquant is built so those outcomes are hard to produce by accident, because the honest order of operations is the only order the pipeline will run.

Belief in a digiquant number rests on process. Research moves through fixed stages, each stage declares what it reads, and every result records the engine and build that produced it, so it can be checked later.

How it works

The unit of work is the whole pipeline. A caller asks for a research task; digiquant plans the stages, runs them through an established backtesting engine and returns the result with its provenance attached.



each arrow is a contract: a stage reads only what the stage before it wrote

That single rule keeps look-ahead bias out: a strategy only sees what earlier stages wrote. The honesty checks are part of the pipeline itself. An estimate the checks refuse comes back empty, with no plausible number to mislead anyone, and a partial result is labelled partial.

Where it fits

digiquant publishes its tasks as tools, so the hub can ask for research without knowing how quant work is ordered. Market history comes in from the data layer; results go out to whoever asked, whether that is the chat, a scheduled session or a dashboard. Model calls go through the shared model layer, and every run is traced, so a Sharpe ratio on a dashboard leads back to the run and engine build behind it.

The pipeline runs wherever compute is cheapest for you: beside the data on one machine for fast backtests, on a timetable in the cloud, or on hardware sized to the workload. The ordering rules travel with it, because they are written into the code itself.

12

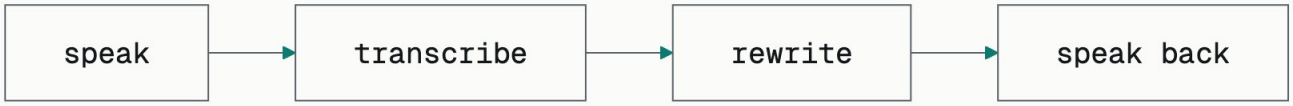
digivoice, the voice loop on your machine

Typing is not always how people think. A thought often arrives spoken and half-formed, while walking or with the hands busy. digivoice catches it: capture speech, transcribe it, reshape the transcript for where it is going and speak results back. The whole loop runs on your own machine, and the audio never leaves it.

The common alternative is to send audio to a hosted service, which trades away the most personal stream a person produces: their voice and everything in the room behind it. digivoice starts from the observation that the machine in front of you is already fast enough. Running locally makes the loop private because there is nowhere else for the audio to go, and it keeps working offline.

How it works

The loop has four stations. Capture records from the microphone to a local audio file. A local speech model turns it into text. An optional rewrite pass reshapes the transcript for its destination, a message, a note or a command, using a small local model. Synthesis reads results back in a local voice.



the loop completes on the machine; only the finished text travels further

The recording stays on disk as audio and every take lands in the history, so a rewrite can be thrown away without losing what was said, and a failed rewrite falls back to the raw transcript. History is an append-only log on the machine that you can search and revisit. Settings can be read and changed from the command line, so other tools can drive the loop, and a built-in checkup reports plainly what the host can and cannot do.

Where it fits

digivoice barely touches the rest of the stack. It never calls the hub in the middle of a loop. When it is done, it pastes ordinary text into whatever is in focus: the chat, a research brief, a note.

It has one home, the machine with the microphone. There is no voice service to deploy, scale or pay for, because the voice was never hosted in the first place.

13

digichat, the interface people touch

Most people never meet the stack. They meet digichat, the chat surface where questions go in and answers stream out. Everything behind it, the hub, the verticals and the models, serves that surface, and the surface itself does very little.

A person signs in, opens a conversation and talks. Answers stream in as they are produced, history is kept per conversation, and artifacts such as documents and visuals render beside the chat rather than inside a wall of text. The same interface can be embedded in another site, so a host can offer the experience to its own visitors with the same backend behind it.

How it works

digichat is two parts that travel together: a request layer and a surface. The request layer signs the person in, works out which conversation they are in, and forwards the message to the hub with a short-lived identity. The surface renders the streaming answer, the history and the artifacts.

Neither half holds anything worth stealing. Sessions are verified on every request. Conversation content is kept in the data layer when one is configured, not in the server. Embedded visitors are authorised per host before they see anything, and a registered host alone authorises nothing: the credential for that visit does.

Where it fits

digichat is the hub's most demanding caller and the data layer's steadiest writer. It relies on the identity service for sessions, on the hub for answers and on the data layer for history. Artifacts render with the same design components as the rest of the surfaces, so the chat looks like part of one product.

Because the interface speaks the standard chat format to the hub, it is also replaceable. Any client that speaks that format and carries a valid identity can sit where digichat sits.

Where it runs

On a laptop the interface runs beside the stack it talks to, which keeps development to a single checkout. In the cloud it runs as a container in front of the hub and verticals. On your own servers it runs wherever you already serve web traffic.

In every home it stays stateless. Stop it, move it or run ten copies, and conversations survive, because the server never held them.

14

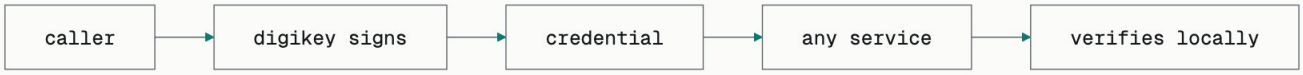
digikey, identity without trust

In a system of a dozen services, the dangerous question is who vouches for whom. If every service trusts every caller, one compromised caller owns everything. digikey is the narrow answer: exactly one service issues the signed credentials services accept, every other service checks them for itself, and no service ever takes a caller at its word.

Two kinds of caller need access. Services and outside callers hold long-lived keys, issued once and shown once, and exchange them for short-lived signed credentials. People sign in through the interface, and the request layer trades their session for the same short-lived credential. Both come from the same issuer.

How it works

Issuing is central and checking is local. The identity service looks at what it is shown, an operator credential, a session or a machine credential, and returns a short-lived signed credential carrying identity, tenant and scopes. Every other service holds the issuer's public signing keys and checks each incoming credential itself: issuer, audience, expiry and scopes. No call home is needed, so if the issuer goes quiet for a few minutes, requests in flight keep working.



one service signs; every other service verifies locally with the public keys

Long-lived credentials are stored only as one-way hashes, so a lost one is replaced, never recovered. Scopes keep every credential narrow: a credential that can read research results cannot spend anything, because it was never granted that scope.

Where it fits

Everything protected meets digikey: the hub, the verticals and the data paths. Each service declares which scopes its routes need, and shared code enforces them the same way everywhere. Outside callers come in through the same exchange, so there is no side door with different rules.

Central issuing is a single point of trust, and the design contains it. Credentials expire quickly, scopes are narrow and keys can be rotated, so a mistake expires before it spreads. Because checking is local, identity works the same on one machine, on your own servers and in a cloud account.

15

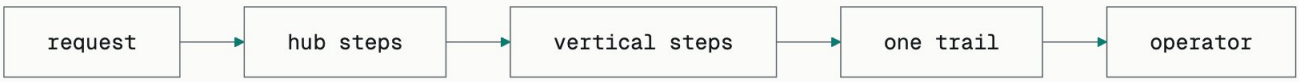
digitrace, seeing what the system did

A system that cannot explain itself cannot be operated. When an answer looks wrong, someone has to be able to ask which tools ran, in what order, what each one saw and how long it took. digitrace is how digithings answers. Once switched on, it records the trail of every request as it happens, so nobody has to piece it together from log lines afterwards.

The trail serves several readers. An operator uses it to debug a bad answer. A reviewer uses it to audit a research number. Anyone deciding whether to trust the system can see the steps behind a result. And once tracing is switched on, the trail shows where to improve: a tool that takes four seconds, or a call that fails one time in ten, stands out at once.

How it works

digitrace has two faces. As a library, it gives every service a thin helper that records when tracing is configured and costs nothing when it is not. Services adopt it without depending on any tracing vendor. As a small service, it reports whether tracing is active and healthy, so the hub and the dashboards can show its status without touching sensitive configuration.



one request, one story: the trail follows it across every service it touches

The trail follows a request across service boundaries, so a step in the hub and the step it triggered in a vertical appear in one story rather than in two logs that have to be matched by timestamp.

Where it fits

Everything that acts is traced. The hub records routing and tool calls, the verticals record their pipeline stages, and the model layer records usage through its hooks. Because the library is vendor-neutral, switching to a different tracing backend changes configuration, not service code.

The trail goes wherever you point it: a backend on your own machine, one beside your cloud workers, or one on your own servers with the retention you choose. Tracing every request costs some storage and a little time per request. That costs far less than debugging a multi-service answer from scattered logs, and you never pay another vendor to watch your own system.

16

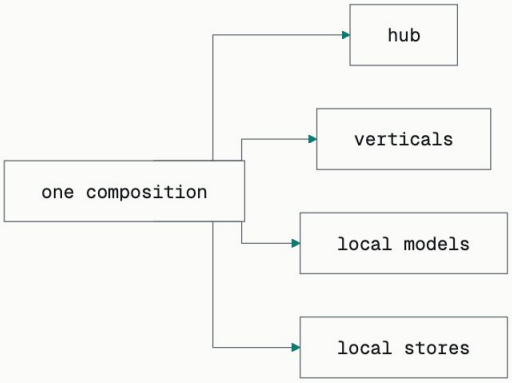
The local stack, one machine

Before the system runs anywhere else, it runs here: one machine, every service beside its neighbours, the full loop from question to answer on hardware you can touch. The local stack is not a demo mode or a trial with features removed. It is the same system arranged for one machine, and for many people it is the only home they will ever need.

A laptop with 16GB of memory is enough: it runs the stack alongside a local model of 7 to 14 billion parameters. Services start side by side, talk over the machine's own network and keep their data in local stores. A developer runs the same composition they ship, and a small firm can run the whole product on one well-chosen computer. Nobody needs an account to begin.

How it works

One declarative file describes which services run, what each needs and how they reach each other. Starting the stack brings them up in dependency order; stopping it puts everything away. Seed documents ship with it and load with one command.



one file describes the whole machine; the services inside are the ones that run everywhere

The services behave exactly as they do in any other home, and a request is signed, checked and traced exactly as in a cloud account, because the code never learns where it is running. The only local concessions are friendlier defaults and warm caches.

Where it fits

The local stack is where the working day happens: development, experiments, demonstrations and small production workloads. The last chapter covers what happens when one machine is no longer enough. Nothing written against the local stack has to be rewritten, so work done locally carries straight over.

17

Start on your machine

The laptop is usually treated as a developer convenience, a smaller and vaguer copy of the real system. That is backwards. If an application cannot run on your own computer, it is hard to trust, hard to debug and hard to hand to anyone else. So the first deployment of anything built on digithings is the computer in front of you.

The first hour

Start the local stack, load the seed documents with one command and ask the chat a question. They answer straight away, so the first thing you see is the whole loop working: the interface, the hub, a model and a trace of what happened. Then connect the model you actually want, local or hosted, and ask again.

The first week

Pick the one use case closest to your work and switch on only what it needs. For research, that is the research vertical and retrieval over your own notes. For documents, retrieval and the vault. For an agent, the tools you already have, described over MCP so the hub can call them. Keep the trace on and read it. It shows how the system reached each answer, which is the fastest way to learn where to change it.

When it outgrows the machine

You will know, because the trace and the machine both tell you: a pipeline that needs more memory, an index that has grown past the disk, more people than one machine should serve. At that point, move the module that is under pressure, not the whole stack. Put it on a server you own or in a cloud account, point its neighbours at the new address and carry on. Its tools, its settings and its credentials work there unchanged.

Location is a decision you make when you have the evidence for it, not before. Open source and open protocols keep that decision yours. Capability and cost go up only as fast as the work does.

Where to go next

The design decisions behind each module, with references into the code, are at digithings.ai/book. The code is open under the MIT licence, and every module carries its own architecture notes. Build something on your own machine first. Then put it where it belongs.

DIGITHINGS

THE DIGITHINGS BOOK

edition 2026 · version 1.0